Some insights for Pitching

▌"10 Luffa bots run a British Crown Court trial — Prosecution argues, Defence fights back, Witnesses testify, a Fact Checker catches lies in real time, 7 AI Jurors deliberate alongside you, and at the end it reveals the truth, scores your reasoning, and tells you which jurors were AI. One command. Fifteen minutes. Five cents."

1. **For "Creative AI" Track**

In VERITAS, AI generation is the entire experience.
Every word of the trial is generated. Every argument is generated. Every juror's personality, every cross-examination, every fact-check, every vote rationale — all generated in real time by 10+ coordinated LLM agents. There is no scripted content. The courtroom exists only because AI creates it, moment by moment, across 14 sequential stages.

Output Quality: Is what the AI produces actually good? Not one generic prompt, — **10 distinct agent personas with role-specific prompting strategies**:

- **Prosecution** builds its case using a "trilogy of proof" framework (motive → means → opportunity) with 3 strategically selected evidence emphasis items that vary per trial
- **Defence** counters with 6 specific reasonable-doubt techniques: timeline challenges, missing evidence, alternative explanations, witness credibility, burden of proof, presumption of innocence
- **Fact Checker** operates at temperature 0.1 with a 70% confidence threshold — it only intervenes when genuinely confident, max 3 times per trial
- **3 juror personas** (Evidence Purist, Sympathetic Doubter, Moral Absolutist) each with full backstories and behavioural instructions that produce genuinely different reasoning styles
- **Witnesses and defendant** are **dynamically generated from case data** — their prompts are built from character profiles, not hardcoded
- A CaseComplexityAnalyzer scores each case (evidence count, type diversity, witness count, timeline complexity) and adjusts every agent's verbosity multiplier (0.8x-1.2x) and argumentation depth ("concise" → "balanced" → "detailed")

User Experience: Is it intuitive and enjoyable?

One command: /start. The 14-state FSM enforces real Crown Court procedure — no UI to learn, no buttons to click. The trial unfolds as a group conversation. Users advance with /continue, deliberate by typing naturally, vote with /vote guilty or /vote not_guilty. The 4-layer reveal creates a dramatic emotional arc that no traditional AI demo achieves.

Originality: Does this feel like something new?

The AI evaluates the human. After the trial, reasoning_evaluator.py scores the user's deliberation statements across 3 axes:
- Evidence reference coverage (did you cite actual evidence? tracked via text matching against all case items)

- Logical fallacy detection (5 regex-based patterns: ad hominem, appeal to emotion, false dichotomy, hasty generalization, straw man)
- Coherence scoring (logical connector usage, statement length, repetition analysis)

This produces a 4-category assessment (sound_correct / sound_incorrect / weak_correct / weak_incorrect) with personalized feedback. **No other generative AI product scores the user's reasoning as its primary output.**

Technical Execution: How well are the models integrated?

- Dual-provider LLM support (OpenAI + Anthropic Claude) with automatic failover
- Token bucket rate limiting tracking both requests/min and tokens/min with sliding windows
- Connection pooling (aiohttp, configurable pool size, DNS caching)
- Exponential backoff retry (3 attempts, configurable delay)
- Graceful degradation: generate_with_fallback() — if LLM fails after all retries, case-specific fallback text keeps the experience running
- Temperature tuning per context: 0.7 creative agents, 0.3 juror voting, 0.1 fact checking
- Optimised Cost: **~$0.05-0.10 per complete 15-minute trial** across all 10 agents

2. For "LuffaNator" (Agentic Track)

10 autonomous Luffa bots coordinate a 14-stage workflow with zero human intervention after /start — the most complex multi-agent system built on Luffa.

Success Metric: Agentic Automation of Real Processes

This is a 10-agent orchestration system.

Each agent has its own Luffa bot identity (UID + secret), appears as a separate participant in the group chat, and speaks only when the state machine triggers its role. The MultiBotService polls ALL configured bots every 1 second, deduplicates messages across bots, filters echo messages, and routes everything through a 14-state FSM.

Concrete agentic behaviours:

1. **Autonomous stage execution** — After /start, the Clerk bot opens proceedings, Prosecution bot presents its case, Defence bot responds, Witness bots testify, Fact Checker bot intervenes when confidence > 70%, Judge bot sums up — all without user commands between stages
2. **Cross-agent context passing** — Prosecution's emphasis items are passed to Defence for counter-arguments; witness testimony is stored and referenced during cross-examination; all deliberation turns are fed into juror vote generation
3. **Real-time fact-checking** — During evidence presentation, the Fact Checker agent independently monitors statements against case evidence and intervenes (max 3 times) when contradictions exceed the 70% confidence threshold
4. **LLM-based autonomous voting** — Each of the 7 AI jurors independently generates a vote by analysing the full deliberation transcript + case evidence at temperature 0.3, returning structured JSON (jury_orchestrator.py, lines 659-791). Fallback heuristics kick in per persona if LLM fails.

Luffa Integration Depth

Direct HTTP architecture — doesn't use the SDK because the luffa-bot-python-sdk has a global robot_key race condition when managing 10+ bots simultaneously. Instead, direct aiohttp POST calls to https://apibot.luffa.im/robot/ endpoints (/receive, /sendGroup).

Hard-won Luffa API handling:
- HTTP 200 on auth failure → checks response code field (not HTTP status)
- Group messages delivered to ALL bots → polls all 10, deduplicates by msgId using OrderedDict
- Echo filtering → compares sender_uid against known bot UIDs to prevent infinite loops
- Type field ambiguity → int() conversion on every type field
- Empty body on send success → graceful handling without JSON parse errors

Multi-user support:
- uid_to_session maps each Luffa user to their active session

- group_users tracks all participants per group
- Session cleanup runs every 5 minutes, 24-hour retention
- Multiple concurrent trials supported across different groups

What Makes It Different From "Simple Chatbots"

1 bot responds to messages	10 bots coordinate across 14 stages
Stateless responses	14-state FSM with timing, persistence, 20-min hard limit
User drives the conversation	Agents autonomously execute the trial sequence
Generic LLM wrapper	Role-specific prompts, complexity-adaptive behaviour, cross-agent context
No outcome	4-layer reveal with reasoning evaluation and juror identity exposure